


```
!git clone https://github.com/Jord65r75i6to8y/Proyecto_de_monitoreo_DE.git
```

```
fatal: destination path 'Proyecto_de_monitoreo_DE' already exists and is not an empty directory.
```

✓ **Análisis de desempeño a realizar**

Analizar el impacto que la reducción de RAM y sistema Operativo generan en el desempeño del juego Don't Starve together Offline.

Variables de respuesta

- FPS
- Frametime
- CPU del proceso
- RAM del proceso
- CPU del sistema
- RAM del sistema
- Swap del sistema
- Actividad de disco (I/O)

Factores controlables y no controlables

Factores controlables y no controlables

Factores controlables

- Cantidad de RAM destinada a la máquina virtual o sistema de pruebas.
- Agresividad del uso de swap en Ubuntu 24.04.
- Cantidad de memoria virtual disponible para swapping.
- Configuración gráfica del juego.
- Cantidad de procesos en segundo plano iniciados por el usuario.
- Versión del juego utilizada durante las pruebas.
- Escenario de prueba ejecutado dentro del juego.

Factores no controlables

- Agresividad del uso de swap en Windows 11.
- Procesos internos del sistema operativo.
- Variaciones en la administración automática de memoria.
- Fragmentación del disco.
- Variabilidad natural de las operaciones de entrada/salida (I/O).
- Diferencias en los algoritmos de administración de memoria entre Windows y Linux.
- Limitaciones para obtener métricas equivalentes de memoria swap asociada a un proceso específico en Windows.

Factores estudiados

• Ideas planteadas

- ¿Como la RAM destinada influye en el desempeño? ¿Que tanto lo hace?
- La reducción de RAM forzará al juego a usar swapping ¿De qué forma la reducción de ram empujará el uso de

memoria virtual?

- ¿Cómo el uso de swapping afecta el desempeño del juego?

- **Factores**

- Influencia de la cantidad de RAM disponible en el desempeño del juego.
- Influencia del uso de memoria virtual (swap) en el desempeño del juego.
- Relación entre reducción de RAM y aumento del uso de swapping durante el gameplay.

Niveles

- **OS**

- Windows 11
- Ubuntu 24.04

- **RAM**

- SL = No limites
- 3.8 GB
- 3.6 GB

Tratamientos

Tratamiento	OS	RAM
T1	Windows	SL GB
T2	Windows	3.8 GB
T3	Windows	3.6 GB
...	...	...

T4	Linux	SL GB
T5	Linux	3.8 GB
T6	Linux	3.6 GB

Monitores a utilizar

Linux (Ubuntu 24.04)

MangoHud (FPS y Frametime)

MangoHud es una herramienta de código abierto diseñada para el monitoreo de videojuegos en tiempo real. Permite visualizar y registrar métricas relacionadas con el desempeño gráfico de una aplicación, incluyendo los fotogramas por segundo (FPS) y los tiempos de fotograma (frametime).

Para este experimento, MangoHud será utilizado para medir las variables de respuesta relacionadas con el desempeño del juego Don't Starve Together, específicamente FPS y frametime.

Psutil (RAM, Swap, CPU y Disco)

Psutil es una biblioteca multiplataforma para Python que permite obtener información detallada sobre procesos y recursos del sistema operativo. La herramienta proporciona acceso a métricas relacionadas con el uso de CPU, memoria RAM, memoria virtual (swap) y actividad de disco.

Durante las pruebas en Ubuntu, Psutil será utilizado para registrar el uso de memoria RAM del proceso del juego, el uso de memoria RAM del sistema, el uso de memoria swap y la actividad de entrada/salida del disco. Estas métricas serán almacenadas automáticamente para su posterior análisis.

Windows 11

MSI Afterburner + RivaTuner Statistics Server (FPS y Frametime)

MSI Afterburner es una herramienta de monitoreo de hardware ampliamente utilizada para la supervisión del rendimiento de videojuegos. En conjunto con RivaTuner Statistics Server (RTSS), permite visualizar y registrar métricas de desempeño en

videojuegos. En conjunto con RivaTuner Statistics Server (RTSS), permite visualizar y registrar métricas de desempeño en tiempo real.

Para este experimento, estas herramientas serán utilizadas para capturar los FPS y frametimes generados durante la ejecución de Don't Starve Together.

Psutil (RAM, Swap, CPU y Disco)

Psutil también será utilizado en Windows 11 con el objetivo de mantener un método de medición consistente entre ambos sistemas operativos. La biblioteca permitirá registrar el uso de memoria RAM del proceso monitoreado, el uso de memoria RAM del sistema, la utilización de memoria virtual (pagefile/swap), el uso de CPU y la actividad de disco durante cada tratamiento experimental.

La utilización de una misma biblioteca en ambos sistemas operativos permite reducir diferencias metodológicas entre tratamientos y facilita la automatización de la captura de datos mediante una única base de código desarrollada en Python.

Consolidación de datos

Las métricas obtenidas mediante las herramientas de monitoreo serán registradas en archivos XLS para su posterior análisis. Los datos serán procesados mediante Python utilizando bibliotecas de análisis de datos como Pandas y herramientas de visualización como Matplotlib.

La utilización de psutil en ambos sistemas operativos permite unificar la captura de las métricas relacionadas con memoria y almacenamiento, reduciendo posibles diferencias derivadas del uso de herramientas distintas para la medición de dichas variables.

✓ Procedimiento de prueba de concepto

Con el objetivo de validar la metodología propuesta para la captura de datos y verificar el correcto funcionamiento de las herramientas de monitoreo, se realizará una prueba de concepto preliminar utilizando únicamente el sistema operativo Ubuntu 24.04. La inclusión de Windows 11 será considerada en etapas posteriores del proyecto una vez que la infraestructura de

captura y procesamiento de datos haya sido validada.

La prueba consistirá en la ejecución de tres corridas independientes con una duración aproximada de cinco minutos por corrida. El propósito de estas ejecuciones es comprobar la correcta recolección de las variables de respuesta, la generación de los archivos de salidas y posibles errores de planteamiento.

Durante cada corrida se seguirá la misma secuencia de actividades dentro del videojuego Don't Starve Together:

1. Iniciar el juego y cargar el mundo seleccionado para la prueba.
2. Caminar por el mapa durante un minuto.
3. Cortar árboles durante dos minutos.
4. Interactuar con criaturas hostiles mediante combate durante un minuto.
5. Abrir el inventario y desplazarse por el mapa durante un minuto.
6. Finalizar la sesión y cerrar el juego.

Durante toda la ejecución se registrarán automáticamente las variables de respuesta definidas para el experimento mediante MangoHud y Psutil. Entre las métricas recolectadas se incluyen FPS, frametime, utilización de CPU, uso de memoria RAM del proceso, uso de memoria RAM del sistema, utilización de memoria swap y actividad de entrada/salida de disco.

Una vez finalizadas las tres corridas, se realizará una revisión de los datos obtenidos con el fin de verificar la consistencia de las mediciones, la correcta sincronización de los registros y la integridad de los archivos generados. Los resultados de esta prueba permitirán determinar si el diseño experimental requiere ajustes antes de proceder con la fase formal de análisis experimental.

Empieza a programar o a [crear código](#) con IA.

```
import pandas as pd
import matplotlib.pyplot as plt

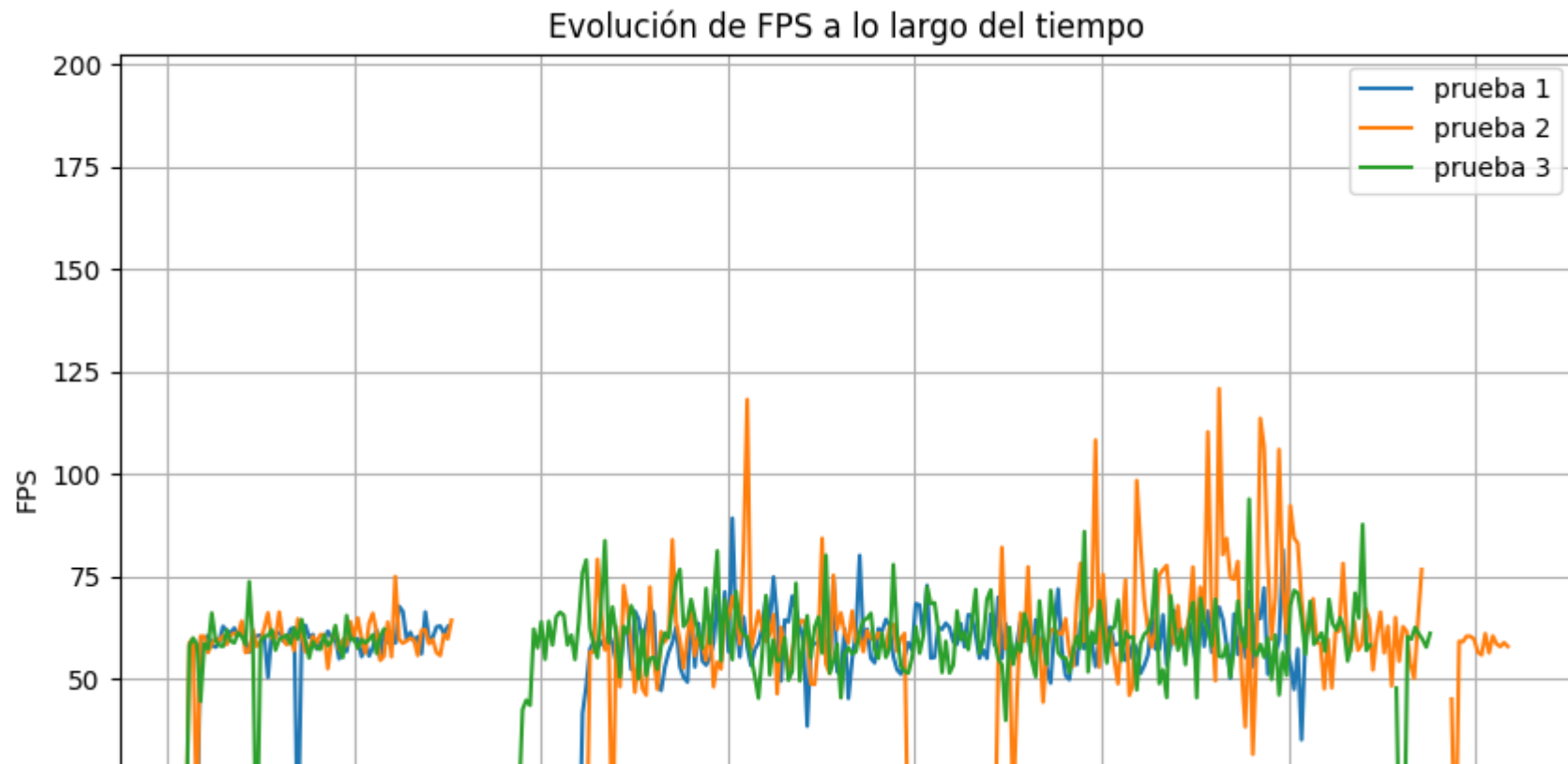
# Cargar los datos desde el archivo Excel
df1 = pd.read_excel("/content/Proyecto_de_monitoreo_DE/pruebas_de_concepto/mergeLinuxPrueba1.xlsx")
df2 = pd.read_excel("/content/Proyecto_de_monitoreo_DE/pruebas_de_concepto/mergeLinuxPrueba2.xlsx")
df3 = pd.read_excel("/content/Proyecto_de_monitoreo_DE/pruebas_de_concepto/mergeLinuxPrueba3.xlsx")
```

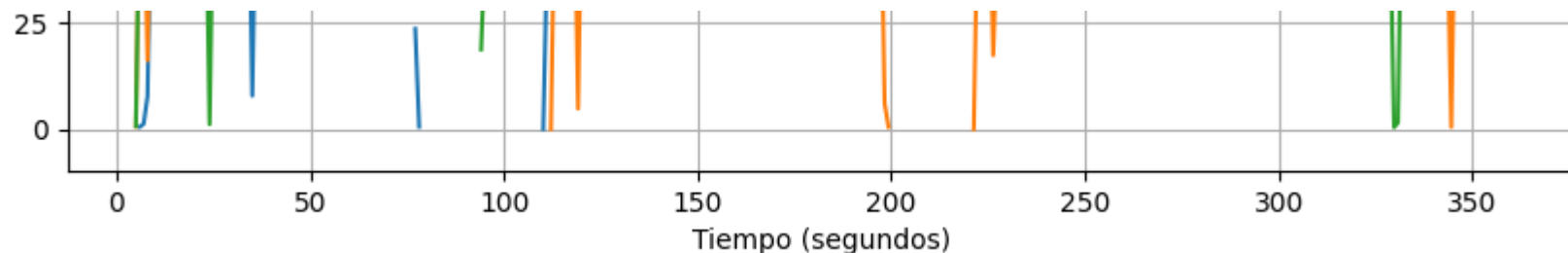
```
plt.figure(figsize=(10, 6))

plt.plot(df1["elapsed"], df1["fps"], label="prueba 1")
plt.plot(df2["elapsed"], df2["fps"], label="prueba 2")
plt.plot(df3["elapsed"], df3["fps"], label="prueba 3")

plt.xlabel("Tiempo (segundos)")
plt.ylabel("FPS")
plt.title("Evolución de FPS a lo largo del tiempo")
plt.legend()
plt.grid(True)

plt.show()
```





Como puede observarse, se presentó una disminución en la tasa de fotogramas por segundo (FPS) entre los segundos 75 y 115 de la ejecución. Esta variación no se debe a una degradación del rendimiento del sistema, sino a una transición propia del videojuego, ocasionada por la muerte del personaje y la posterior aparición de la pantalla de carga. Por este motivo, dicho comportamiento se considera un evento particular de la prueba y no representa el desempeño normal del sistema. Fuera de este intervalo, los datos muestran un comportamiento estable y representativo para el análisis.

```
import pandas as pd
import matplotlib.pyplot as plt

# Cargar los datos desde el archivo Excel

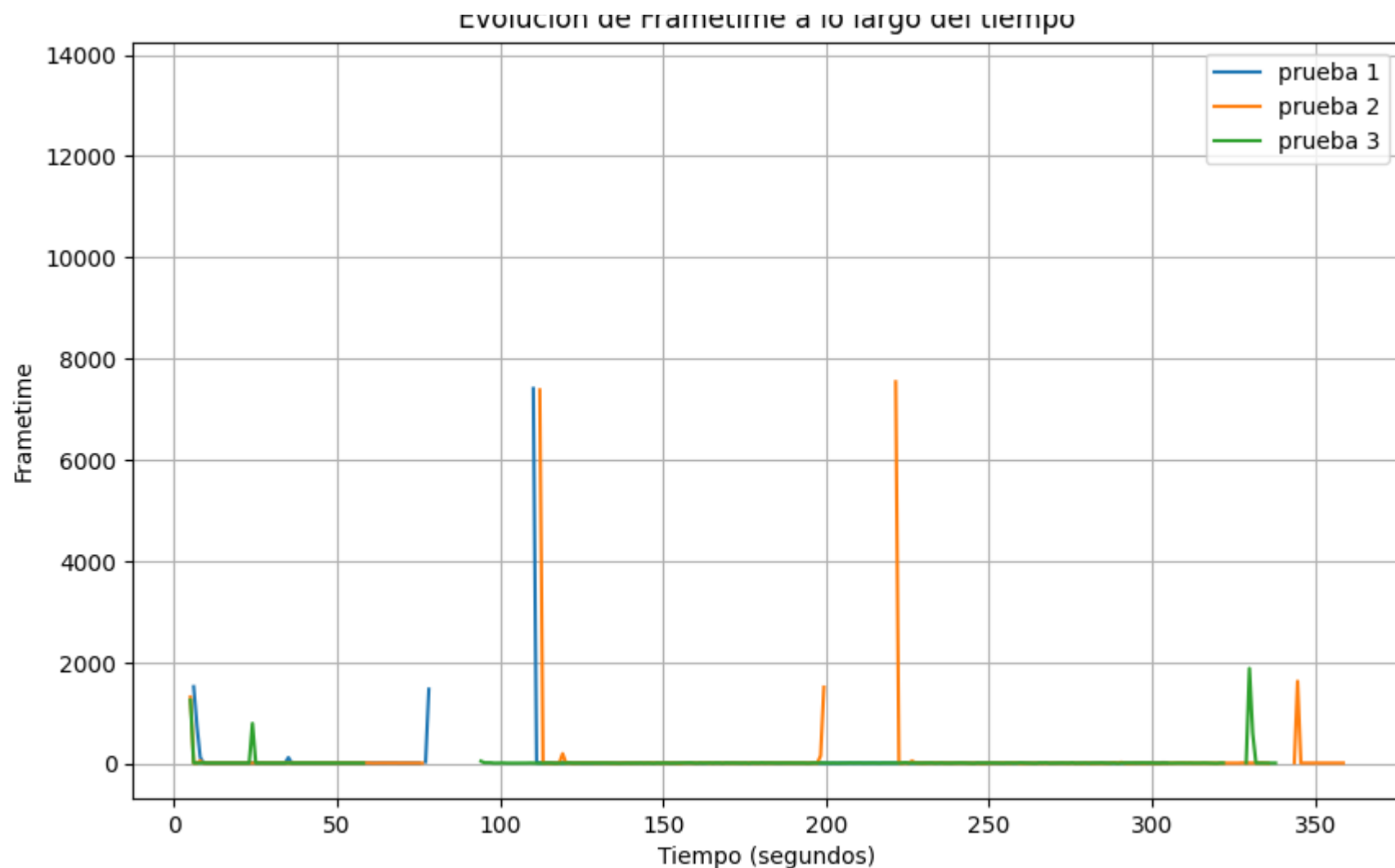
plt.figure(figsize=(10, 6))

plt.plot(df1["elapsed"], df1["frametime"], label="prueba 1")
plt.plot(df2["elapsed"], df2["frametime"], label="prueba 2")
plt.plot(df3["elapsed"], df3["frametime"], label="prueba 3")

plt.xlabel("Tiempo (segundos)")
plt.ylabel("Frametime")
plt.title("Evolución de Frametime a lo largo del tiempo")
plt.legend()
plt.grid(True)

plt.show()
```

Evolución de Frametime a lo largo del tiempo



Como puede deducirse del gráfico anterior, se observa una ausencia de datos de frametime entre los segundos 75 y 115 de la ejecución. Este comportamiento se debe a las mismas razones descritas para el gráfico de FPS, ya que durante dicho intervalo el personaje murió y el juego pasó a una pantalla de carga antes de reanudar la partida.

Por este motivo, la ausencia de registros en ese intervalo se considera un evento particular de la prueba y no representa el comportamiento normal del sistema ni del videojuego. Fuera de este período, los datos muestran una tendencia relativamente

estable y representativa para el análisis.

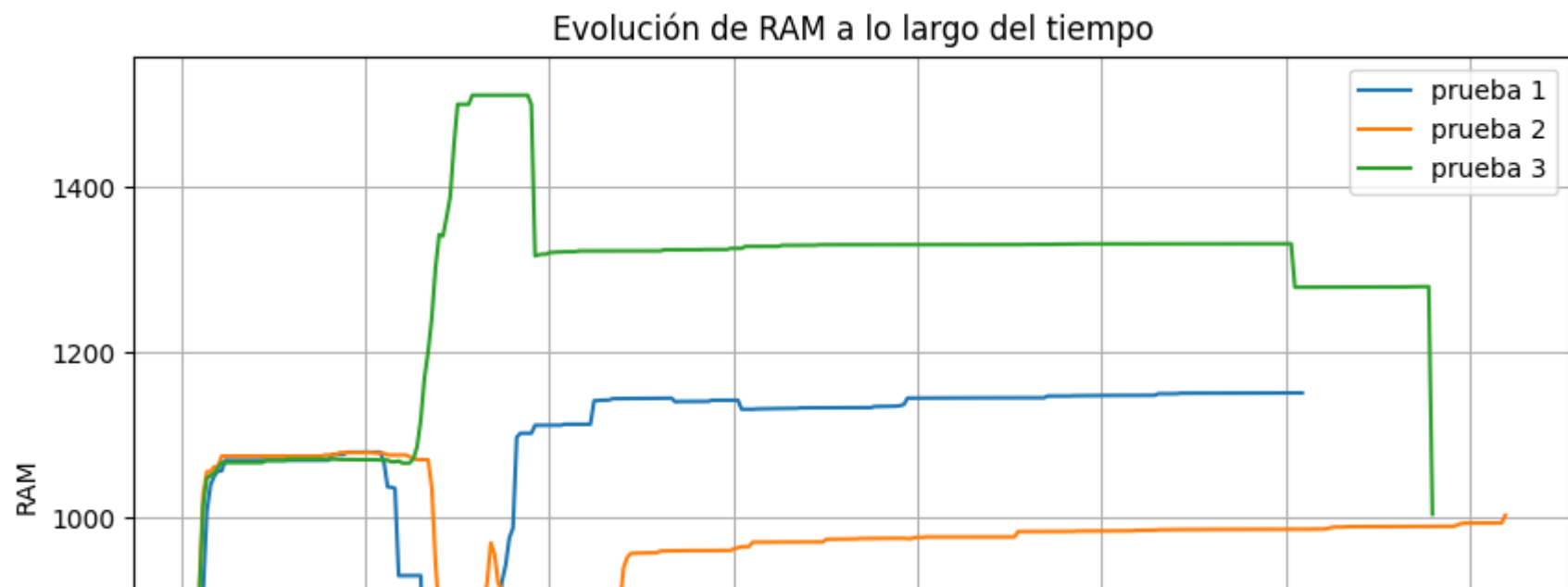
```
import pandas as pd
import matplotlib.pyplot as plt

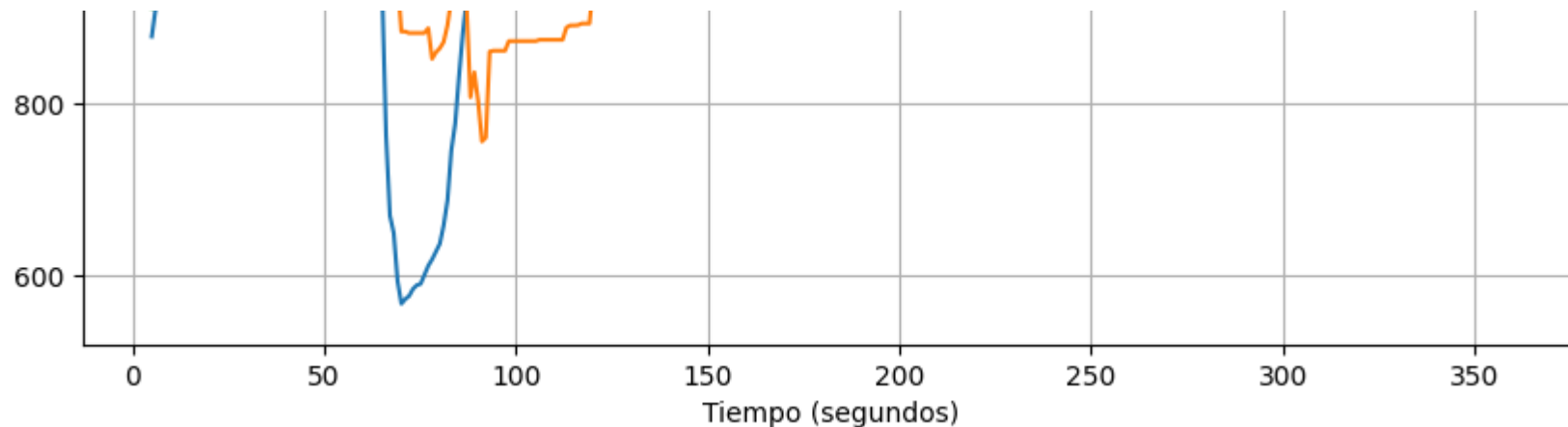
plt.figure(figsize=(10, 6))

plt.plot(df1["elapsed"], df1["RAM"], label="prueba 1")
plt.plot(df2["elapsed"], df2["RAM"], label="prueba 2")
plt.plot(df3["elapsed"], df3["RAM"], label="prueba 3")

plt.xlabel("Tiempo (segundos)")
plt.ylabel("RAM")
plt.title("Evolución de RAM a lo largo del tiempo")
plt.legend()
plt.grid(True)

plt.show()
```





El gráfico muestra una variación considerable en el consumo de memoria RAM a lo largo de las tres corridas. Las pruebas 1 y 2 presentan un comportamiento similar, debido a que ambas fueron afectadas por la muerte del jugador. Por el contrario, la tercera corrida alcanzó una mayor duración, lo que permitió observar el comportamiento completo del sistema durante la ejecución del escenario experimental, incluyendo la creación del mundo y la interacción dentro del mismo.

Durante las pruebas, el consumo de memoria registrado a nivel del proceso principal del juego mediante Psutil osciló aproximadamente entre 400 MB y 1550 MB. Sin embargo, al considerar la memoria total del cgroup asociado a la ejecución del juego y su entorno (Steam Runtime, bibliotecas y procesos auxiliares), se observaron valores significativamente mayores, con un rango aproximado entre 3400 MB y 4500 MB. Esto evidencia que el consumo real de memoria del sistema durante la ejecución es sustancialmente mayor al observado únicamente en el proceso principal.

Con base en los resultados obtenidos durante la prueba de concepto, se determinó ajustar los niveles del factor memoria RAM para la fase de análisis experimental. Las pruebas iniciales con límites más restrictivos provocaron terminaciones prematuras del proceso debido a la activación del OOM Killer durante la carga del mundo, lo que impidió la recolección completa de las variables de respuesta. En consecuencia, los niveles fueron redefinidos considerando el comportamiento estable del sistema observado mediante cgroups.

Los tratamientos definidos para la fase experimental serán los siguientes:

- NO: Sin restricción de memoria.

- N1: Límite de memoria RAM de 3600 MB.
- N2: Límite de memoria RAM de 3800 MB.

La selección de estos niveles busca generar condiciones de operación cercanas al umbral real de consumo del sistema, permitiendo evaluar el impacto de la disponibilidad de memoria sobre las variables de respuesta consideradas en el experimento, tales como FPS, frametime, uso de CPU, utilización de memoria RAM del sistema y actividad de entrada/salida del sistema.

✓ Conclusiones de la prueba de concepto

Las pruebas de concepto realizadas permitieron verificar el correcto funcionamiento de las herramientas de monitoreo seleccionadas y la metodología de captura de datos propuesta. Se comprobó la generación adecuada de los archivos de salida y la recolección de las variables de respuesta definidas para el experimento, incluyendo FPS, frametime, utilización de CPU, uso de memoria RAM, utilización de memoria swap y actividad de entrada/salida de disco.

Asimismo, los resultados obtenidos permitieron identificar el rango de consumo de memoria del videojuego durante las sesiones de prueba, lo que llevó al ajuste de los niveles de restricción de memoria RAM que serán utilizados en la fase de análisis experimental.

Con base en estos resultados, se considera que la metodología propuesta es adecuada para continuar con la ejecución formal de los tratamientos experimentales.

```
import pandas as pd
import matplotlib.pyplot as plt

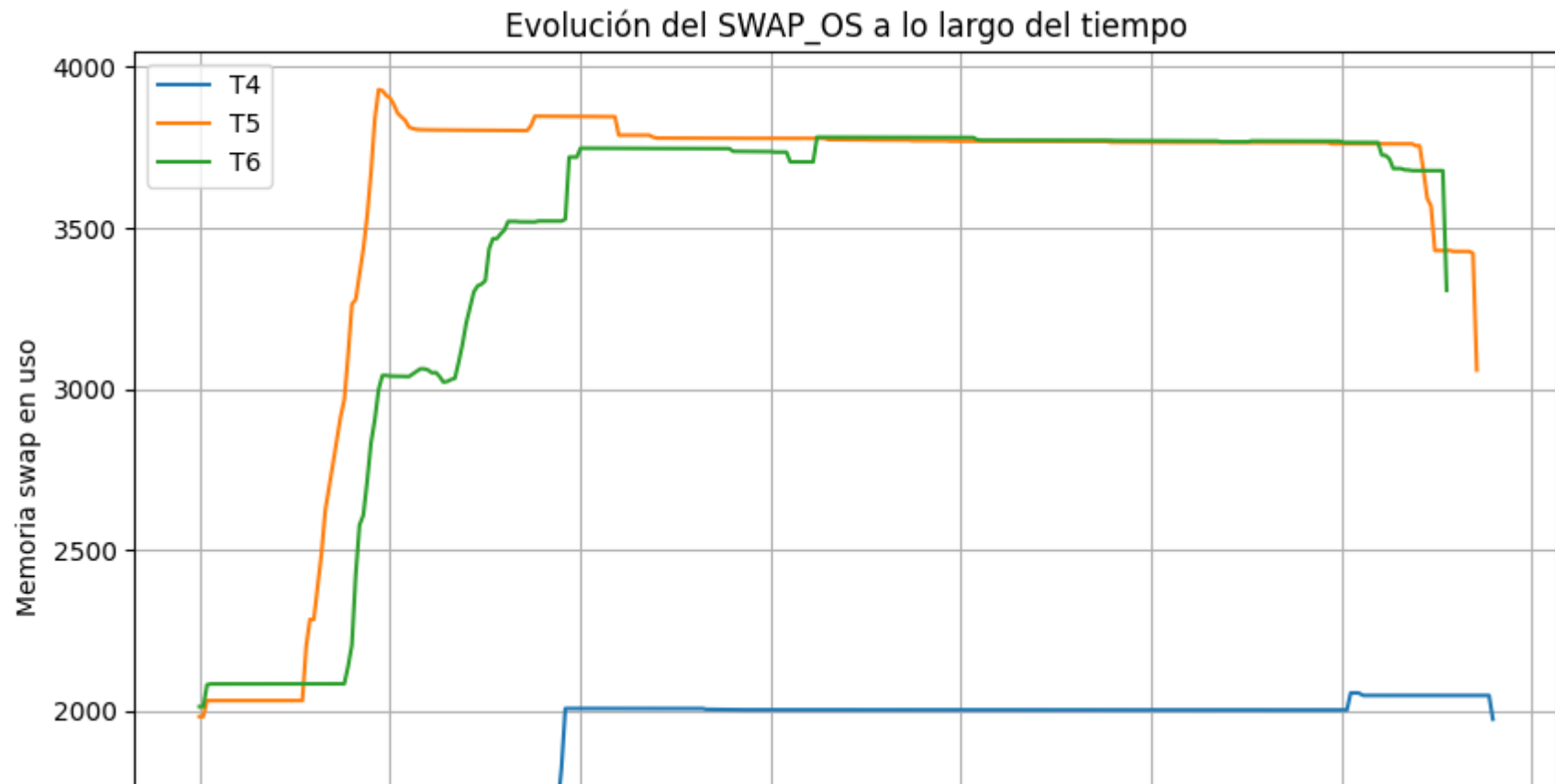
# Cargar los datos desde el archivo Excel
df1 = pd.read_excel("/content/Proyecto_de_monitoreo_DE/pruebas_de_concepto/mergeLinuxPrueba3.xlsx")
df2 = pd.read_excel("/content/Proyecto_de_monitoreo_DE/pruebas_de_concepto/mergeLinuxT3600.xlsx")
df3 = pd.read_excel("/content/Proyecto_de_monitoreo_DE/pruebas_de_concepto/mergeLinuxT3800.xlsx")
```

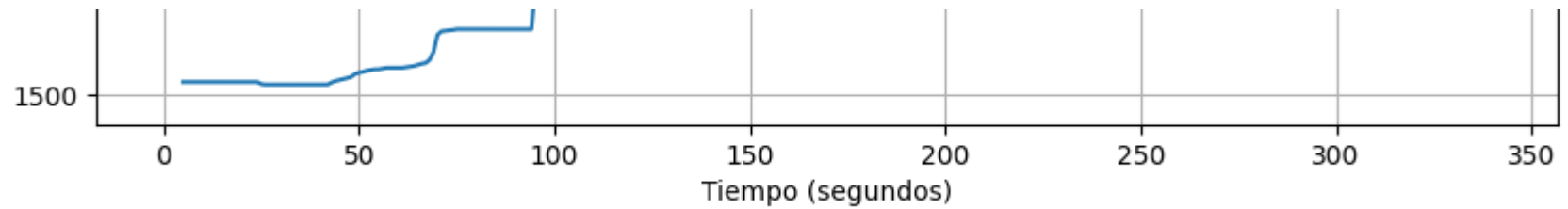
```
plt.figure(figsize=(10, 6))

plt.plot(df1["elapsed"], df1["SWAP_OS"], label="T4")
plt.plot(df2["elapsed"], df2["SWAP_OS"], label="T5")
plt.plot(df3["elapsed"], df3["SWAP_OS"], label="T6")

plt.xlabel("Tiempo (segundos)")
plt.ylabel("Memoria swap en uso")
plt.title("Evolución del SWAP_OS a lo largo del tiempo")
plt.legend()
plt.grid(True)

plt.show()
```





```
import pandas as pd
import matplotlib.pyplot as plt

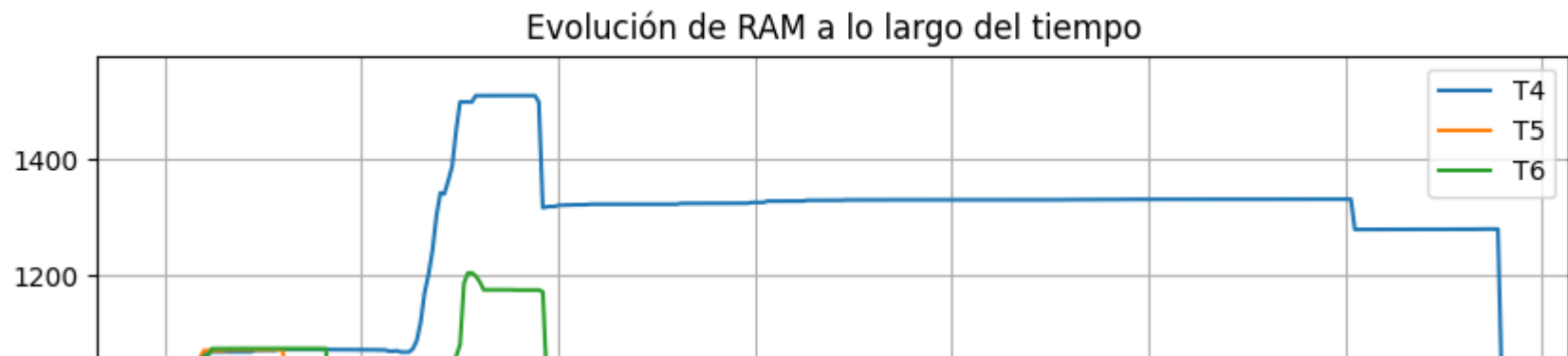
# Cargar los datos desde el archivo Excel

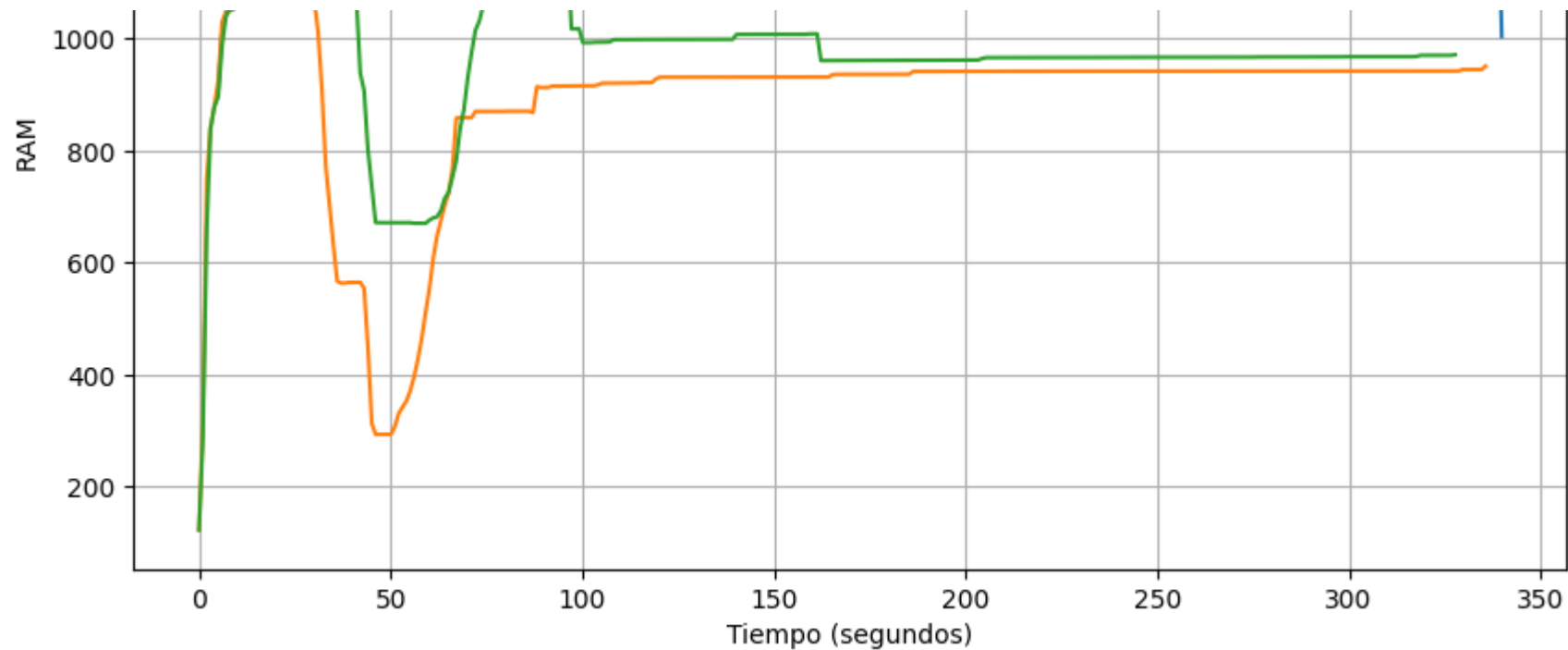
plt.figure(figsize=(10, 6))

plt.plot(df1["elapsed"], df1["RAM"], label="T4")
plt.plot(df2["elapsed"], df2["RAM"], label="T5")
plt.plot(df3["elapsed"], df3["RAM"], label="T6")

plt.xlabel("Tiempo (segundos)")
plt.ylabel("RAM")
plt.title("Evolución de RAM a lo largo del tiempo")
plt.legend()
plt.grid(True)

plt.show()
```





Los dos gráficos de líneas anteriores nos permiten hacernos una idea de las distintas relaciones que pueden representarse entre las variables analizadas. Esto resulta útil tanto para identificar tendencias y comportamientos durante el experimento como para explorar nuevas relaciones que podrían ser de interés en futuras investigaciones. Asimismo, estos gráficos pueden aportar información valiosa para el análisis inmediato de los resultados y facilitar la interpretación del comportamiento del sistema bajo los diferentes tratamientos.

A continuación, se presentan las estadísticas descriptivas obtenidas de manera preliminar para las pruebas realizadas. En particular, se calcularon los valores promedio de las principales métricas de rendimiento registradas durante cada tratamiento, incluyendo los fotogramas por segundo (FPS), el tiempo por fotograma (frametime), el consumo de memoria RAM del proceso, el uso de CPU y el consumo de memoria de intercambio (swap) del sistema operativo. Estas estadísticas permiten realizar una primera comparación entre los distintos tratamientos y sirven como base para el análisis de los resultados obtenidos.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt

df1 = pd.read_excel("mergeLinuxPrueba3.xlsx")
df2 = pd.read_excel("mergeLinuxT3600.xlsx")
df3 = pd.read_excel("mergeLinuxT3800.xlsx")

df1["tratamiento"] = "T4"
df2["tratamiento"] = "T5"
df3["tratamiento"] = "T6"

df = pd.concat([df1, df2, df3], ignore_index=True)

promedios = df.groupby("tratamiento").agg({
    "fps": "mean",
    "frametime": "mean",
    "RAM": "mean",
    "CPU": "mean",
    "SWAP_OS": "mean"
}).reset_index()

print(promedios)
```

	tratamiento	fps	frametime	RAM	CPU	SWAP_OS
0	T4	58.711436	135.444135	1285.112978	83.410149	1897.661532
1	T5	57.988487	124.611154	892.741455	85.353869	3574.886858
2	T6	57.512046	171.703574	971.413491	83.691768	3452.706210

✓ Diseño final del experimento

Para este último avance me dediqué principalmente a consolidar y automatizar las herramientas utilizadas durante la recolección y el procesamiento de los datos experimentales.

Inicialmente, ya contaba con las herramientas necesarias para obtener las mediciones en Linux. Estas consistían en el script **metricasPsutil.py**, encargado de registrar métricas del sistema, y en **mergeLinux.py** (actualmente renombrado como **mergeMetricas.py**) cuya función era combinar los archivos generados por **metricasPsutil.py** con los archivos producidos por

`mergeMetrics.py`, cuya función era combinar los archivos generados por `metricsLinux.py` con los archivos producidos por MangoHud, que contienen las métricas de FPS y *frametime*. La unión de ambos conjuntos de datos se realizaba utilizando la columna **elapsed** como referencia temporal.

En su versión original, **mergeLinux.py** requería que el usuario indicara manualmente cada par de archivos y la ruta donde se almacenaría el resultado, lo que hacía que el proceso fuera lento y propenso a errores cuando se trabajaba con un gran número de mediciones. Por esta razón, el script fue modificado para que procesara automáticamente todos los archivos contenidos en una carpeta. Para ello, primero identifica cuáles corresponden a **metricsPsutil.py** y cuáles a MangoHud; posteriormente, utilizando la fecha y hora de inicio del monitoreo presentes en el nombre de los archivos, empareja automáticamente aquellos cuya diferencia temporal es mínima. Finalmente, aplica una corrección sobre la columna **elapsed** para compensar el desfase existente entre ambas herramientas y realizar la combinación de los datos de forma adecuada.

Posteriormente, el script también fue adaptado para permitir la integración de mediciones provenientes de Windows, utilizando los registros generados por MSI Afterburner. Como parte de este proceso, se desarrolló el programa **MSIAfterBurnerToCSV.py**, el cual convierte los archivos HML generados por MSI Afterburner al formato CSV y, además, automatiza parte del proceso de recolección de métricas durante las pruebas.

Durante la integración de los datos obtenidos en Windows surgieron algunos inconvenientes relacionados con diferencias de formato respecto a los archivos generados en Linux. Para solucionarlos se desarrolló el script **correcciones.py**, encargado de corregir inconsistencias detectadas, como el uso de la columna **elapse** en lugar de **elapsed** y diferencias en el formato de la fecha presente en los nombres de los archivos.

Uno de los principales desafíos del experimento fue la limitación de memoria en Windows. A diferencia de Linux, donde es posible utilizar *cgroups* para restringir la memoria disponible para un proceso y sus procesos hijos, Windows no ofrece un mecanismo equivalente de uso tan directo. Debido al tiempo disponible para el proyecto y a que no contaba con experiencia previa en el lenguaje C# ni con la API de bajo nivel *Job Objects*, recurrí al apoyo de ChatGPT para desarrollar un script que permitiera establecer dichos límites de memoria. Esta herramienta resultó de gran utilidad, ya que permitió implementar la funcionalidad requerida y continuar con las pruebas experimentales sin necesidad de desarrollar una solución desde cero.

```
from pathlib import Path
import pandas as pd
```

```
resumen = []

tratamientos = {
    "Linux SL": "/content/Proyecto_de_monitoreo_DE/treatment_Linux/treatment_SL/mergeSL",
    "Linux 3600": "/content/Proyecto_de_monitoreo_DE/treatment_Linux/treatment_3600/merge3600",
    "Linux 3800": "/content/Proyecto_de_monitoreo_DE/treatment_Linux/treatment_3800/merge3800",

    "Windows SL": "/content/Proyecto_de_monitoreo_DE/treatment_Windows/treatment_SL/mergeSL",
    "Windows 3600": "/content/Proyecto_de_monitoreo_DE/treatment_Windows/treatment_3600/merge3600",
    "Windows 3800": "/content/Proyecto_de_monitoreo_DE/treatment_Windows/treatment_3800/merge3800",
}

for tratamiento, carpeta in tratamientos.items():

    for archivo in Path(carpeta).glob("*_merge.csv"):

        df = pd.read_csv(archivo)

        resumen.append({
            "tratamiento": tratamiento,
            "fps": df["fps"].mean(),
            "RAM": df["RAM"].mean(),
            "CPU": df["CPU"].mean(),
            "frametime": df["frametime"].mean(),
            "SWAP": df["SWAP_OS"].mean(),
            "I/O_output": df["I/O_output"].mean(),
            "I/O_input": df["I/O_input"].mean(),
        })

df_resumen = pd.DataFrame(resumen)

df_resumen_comprimido = df_resumen.groupby("tratamiento").agg({
    "fps": "mean",
    "RAM": "mean",
    "CPU": "mean",
    "frametime": "mean",
```

```
"SWAP": "mean"
}).reset_index()

display(df_resumen_comprimido)
```

	tratamiento	fps	RAM	CPU	frametime	SWAP
0	Linux 3600	58.680813	949.574985	86.994169	139.018659	2886.596669
1	Linux 3800	58.847840	958.164148	86.854264	136.233670	2316.678836
2	Linux SL	59.236745	1196.935231	84.519278	139.607713	2721.075874
3	Windows 3600	42.697474	1204.038759	125.549732	184.836584	1878.311186
4	Windows 3800	42.120619	1080.998174	132.404992	200.434740	2240.984951
5	Windows SL	44.318896	976.284554	127.138749	166.857262	1873.263241

```
import matplotlib.pyplot as plt

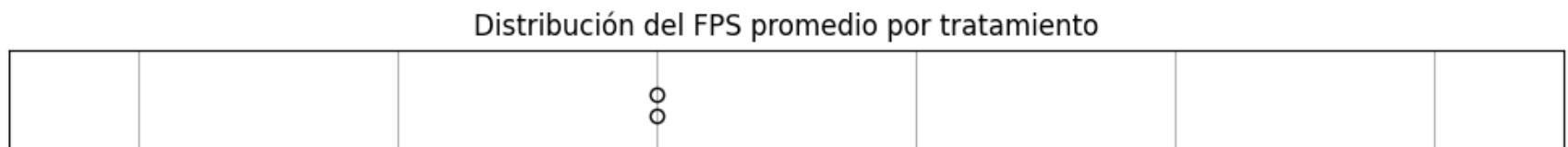
df_resumen.boxplot(column="fps", by="tratamiento", figsize=(12,8))

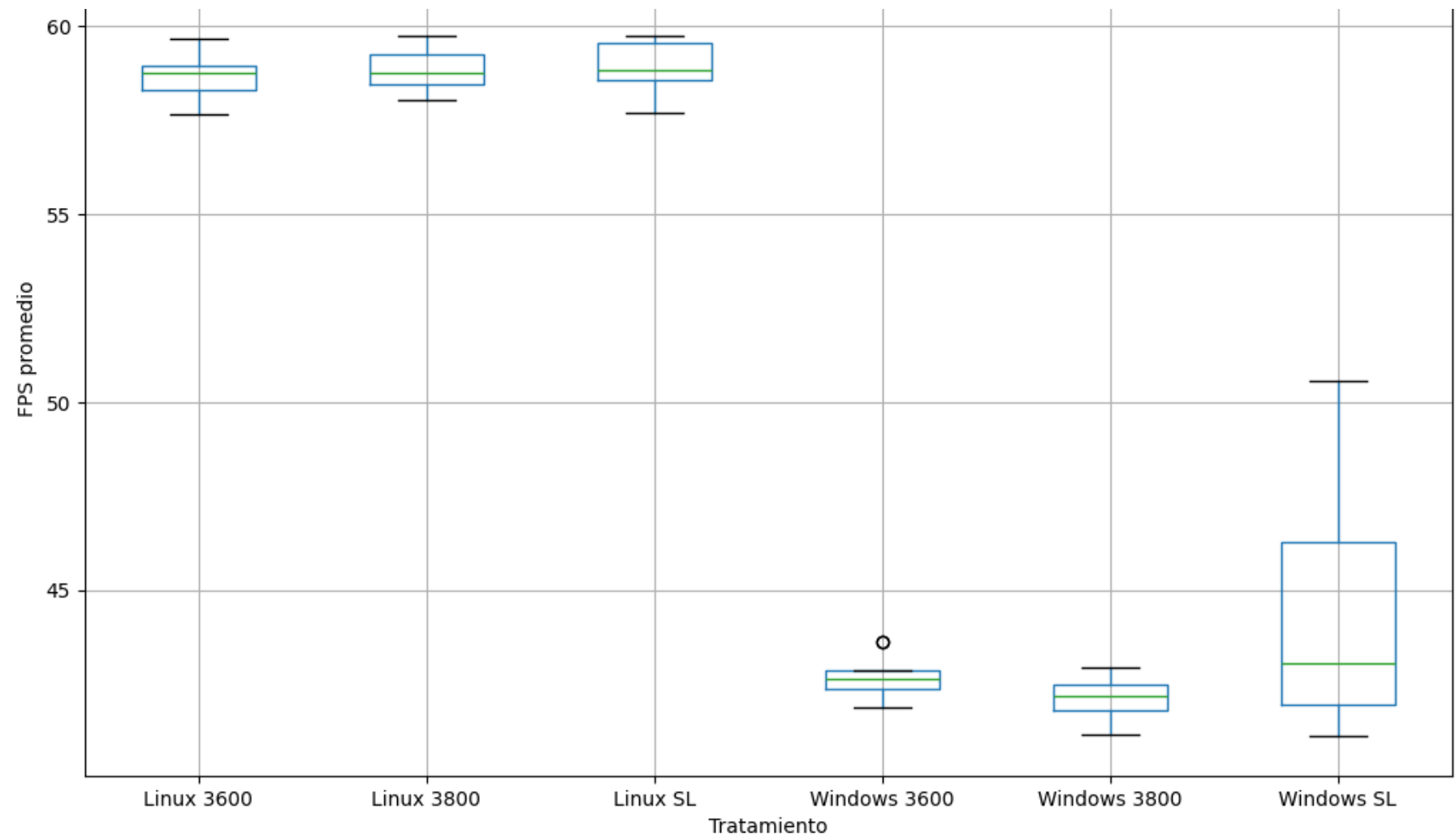
plt.title("Distribución del FPS promedio por tratamiento")
plt.suptitle("")

plt.xlabel("Tratamiento")
plt.ylabel("FPS promedio")

plt.grid(True)

plt.show()
```





Estas primeras escaramuzas con los datos muestran lo que ya suponíamos inicialmente: en lo que respecta a mantener FPS altos, Ubuntu sigue mostrando un mejor desempeño que Windows 11 bajo las condiciones evaluadas. La diferencia observada llega incluso a alcanzar aproximadamente los 10 FPS entre ambos sistemas operativos

loga incluso a alcanzar aproximadamente los 10 FPS entre ambos sistemas operativos.

Esto sugiere que Ubuntu logra manejar de mejor manera los recursos necesarios para la ejecución del juego, permitiendo mantener una mayor tasa de cuadros. Sin embargo, esta es una primera observación basada principalmente en FPS

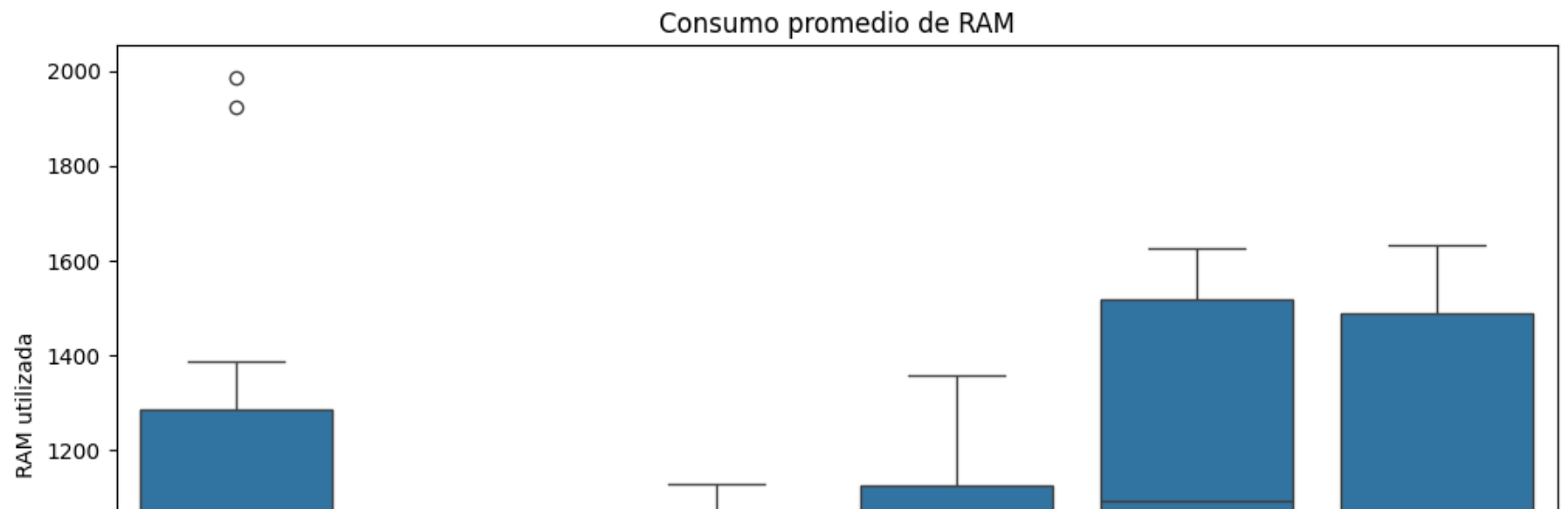
```
import seaborn as sns

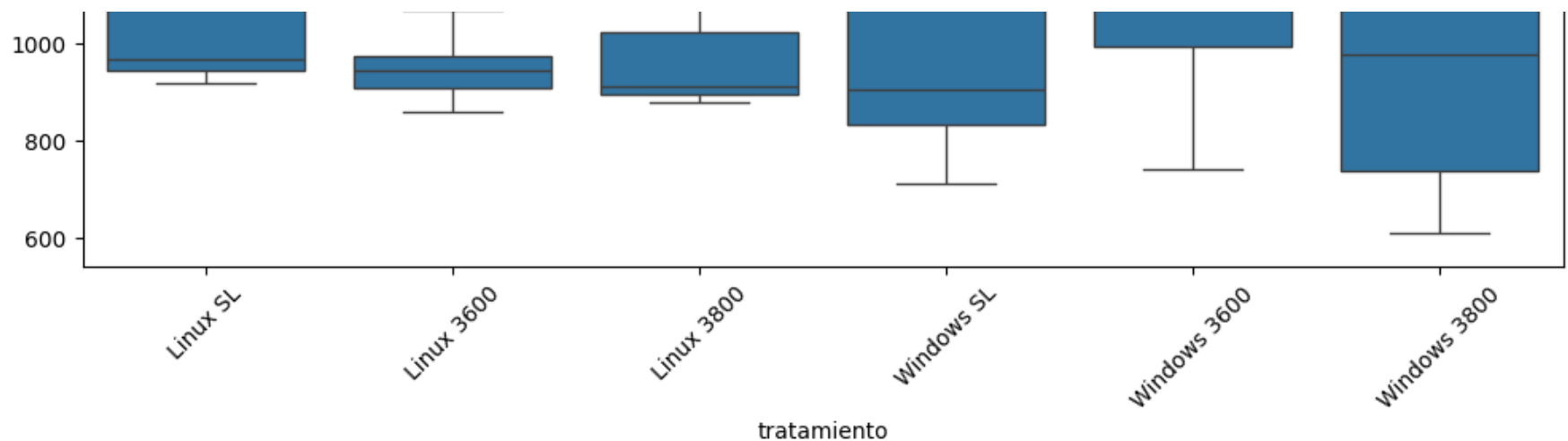
plt.figure(figsize=(12,6))

sns.boxplot(
    data=df_resumen,
    x="tratamiento",
    y="RAM"
)

plt.xticks(rotation=45)
plt.title("Consumo promedio de RAM")
plt.ylabel("RAM utilizada")

plt.show()
```





Ahora bien, esto resulta interesante. En lo que respecta a Linux, parece que mientras más reducíamos la memoria RAM disponible, menor era el uso de memoria observado por el sistema. En cambio, Windows presentó un comportamiento contrario: mientras más reducíamos la memoria disponible, mayor parecía ser el uso de RAM por parte del proceso.

Este comportamiento resulta llamativo, ya que podría indicar diferencias en la forma en que ambos sistemas operativos administran la presión de memoria. Una posible explicación es que Windows intenta mantener más información del juego en memoria física mientras sea posible, evitando recurrir al archivo de paginación hasta que sea necesario.

También podría existir un comportamiento propio de Steam o del motor del juego, donde al detectar suficiente memoria disponible intenta mantener más recursos cargados para evitar accesos posteriores al disco. En este caso, el aumento del uso de RAM no necesariamente representaría un problema, sino una estrategia para mejorar tiempos de acceso y mantener estabilidad en la ejecución.

```
import seaborn as sns
```

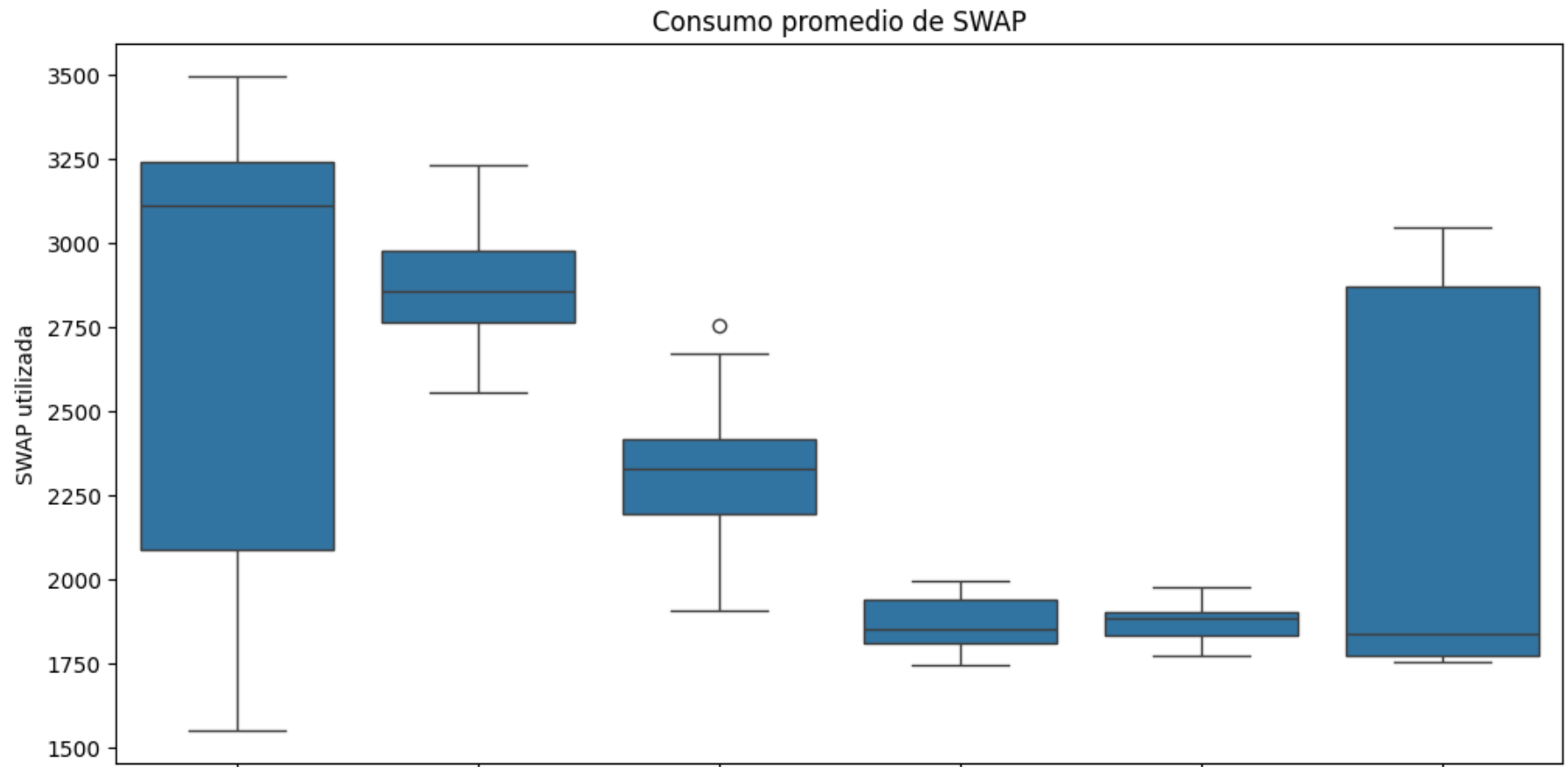
```
plt.figure(figsize=(12, 6))
```

```
plt.figure(figsize=(12,6))

sns.boxplot(
    data=df_resumen,
    x="tratamiento",
    y="SWAP"
)

plt.xticks(rotation=45)
plt.title("Consumo promedio de SWAP")
plt.ylabel("SWAP utilizada")

plt.show()
```



Linux 5L

Linux 3600

Linux 3800

Windows 5L

Windows 3600

Windows 3800

tratamiento

Esta es la observación más llamativa hasta el momento. En el caso de Linux, cuando no existía ninguna limitación parecía utilizar una cantidad considerable de memoria virtual o swap en comparación con los tratamientos donde se fue reduciendo la memoria disponible. Sin embargo, al aplicar el límite de 3800 MB, el uso de memoria virtual disminuyó, mientras que al reducir nuevamente la memoria disponible a 3600 MB volvió a aumentar, llegando incluso a superar el tratamiento sin límites.

Este comportamiento resulta interesante debido a que no sigue un patrón completamente lineal, donde una menor cantidad de memoria disponible implique necesariamente un aumento constante del uso de swap. Parece existir un punto intermedio donde el comportamiento del sistema cambia y la administración de memoria toma una estrategia diferente.

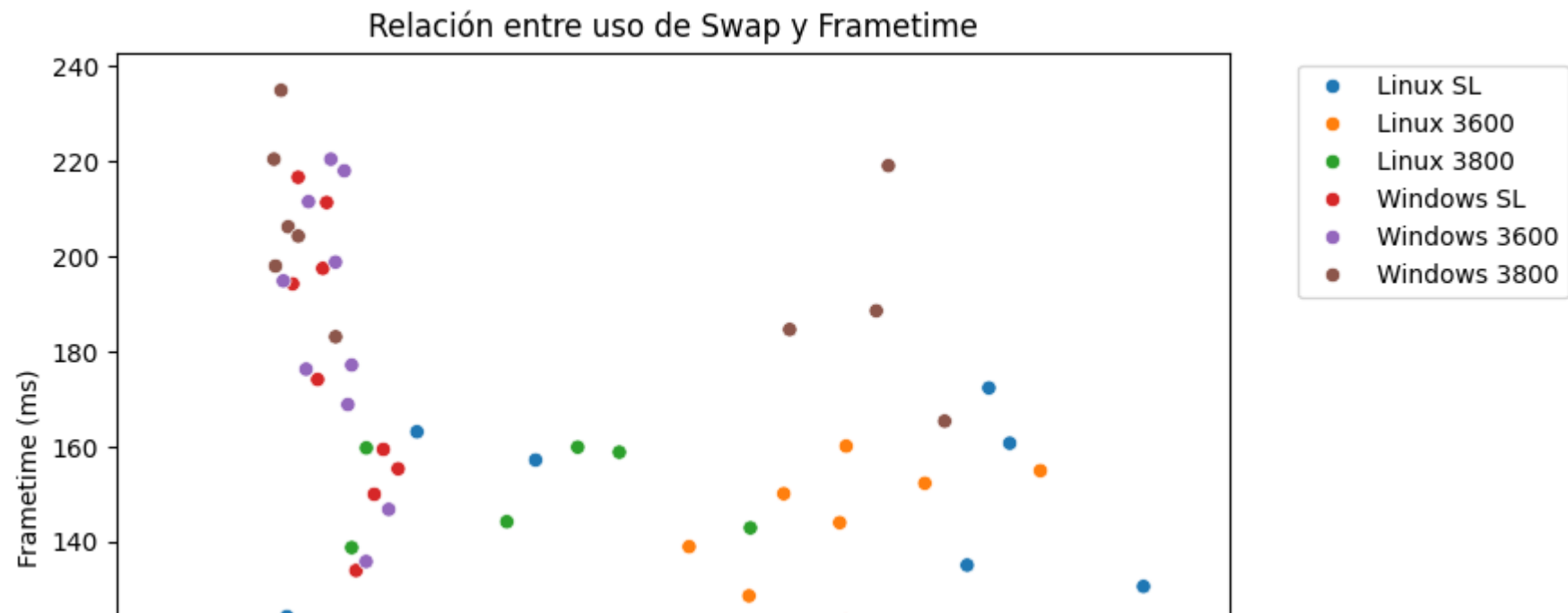
Por otro lado, Windows presentó un comportamiento diferente. Tanto el tratamiento sin límites como el de 3600 MB mantuvieron un uso relativamente bajo de memoria virtual; sin embargo, el caso de 3800 MB resulta precisamente el más extraño, ya que siendo un tratamiento intermedio terminó utilizando más memoria virtual que cualquiera de los otros escenarios.

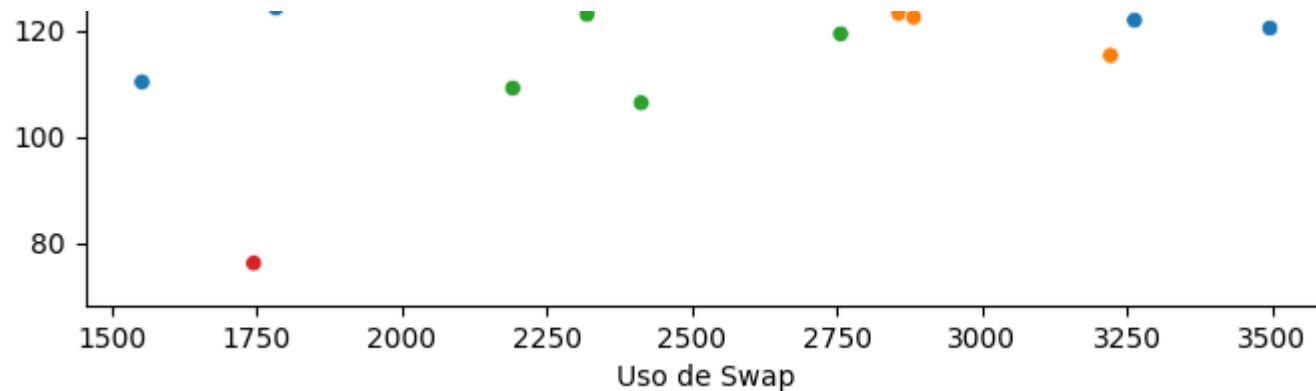
Estos resultados sugieren que la relación entre la memoria asignada, el uso de RAM y la utilización de memoria virtual depende de la estrategia de administración de memoria del sistema operativo. No necesariamente una mayor restricción provoca siempre mayor uso de swap, sino que puede existir un punto donde el sistema cambia la forma en que conserva, libera o pagina información.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,6))
```

```
ax = sns.scatterplot(  
    data=df_resumen,  
    x="SWAP",  
    y="frametime",  
    hue="tratamiento"  
)  
  
ax.legend(  
    bbox_to_anchor=(1.05, 1),  
    loc="upper left"  
)  
  
plt.title("Relación entre uso de Swap y Frametime")  
plt.xlabel("Uso de Swap")  
plt.ylabel("Frametime (ms)")  
  
plt.show()
```





Este gráfico nos termina por mostrar una relación interesante: bajo las condiciones evaluadas, el uso de swap parece estar asociado con mejores valores de frametime. Un ejemplo claro se observa en Windows, donde parece existir una fuerte tendencia a mantener la mayor cantidad posible de información en RAM y reducir el uso de memoria virtual; sin embargo, este comportamiento parece haber tenido un costo reflejado en el frametime obtenido.

Esto resulta llamativo, ya que normalmente se espera que el uso de memoria virtual tenga un impacto negativo debido a la diferencia de velocidad entre la RAM y el almacenamiento secundario. No obstante, los resultados sugieren que una política más agresiva de mantener todo en memoria física no necesariamente garantiza un mejor rendimiento, ya que puede aumentar la presión sobre la RAM disponible.

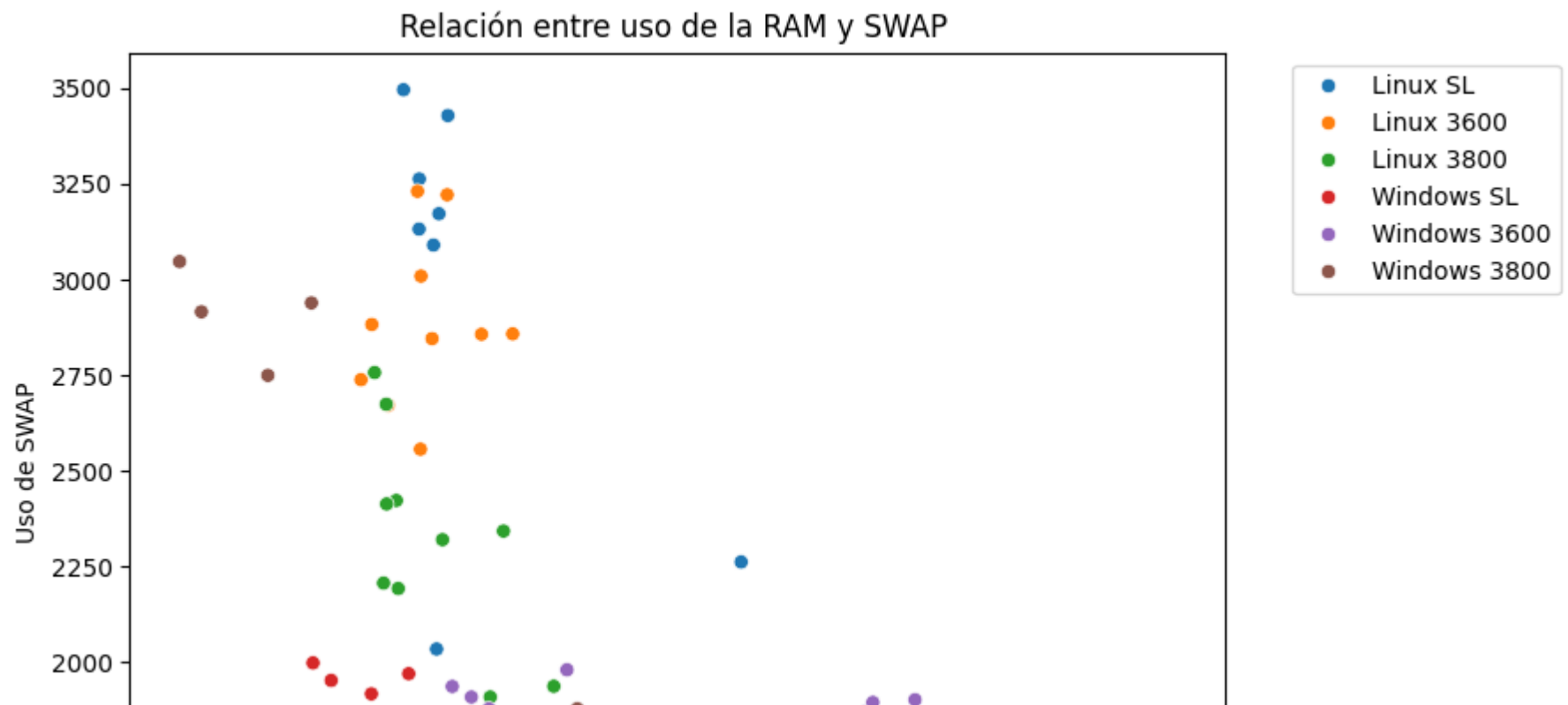
Por lo tanto, en este experimento el uso de swap no debe interpretarse únicamente como una penalización, sino como una herramienta de administración de memoria que puede ayudar al sistema a distribuir mejor los recursos dependiendo del nivel de presión existente.

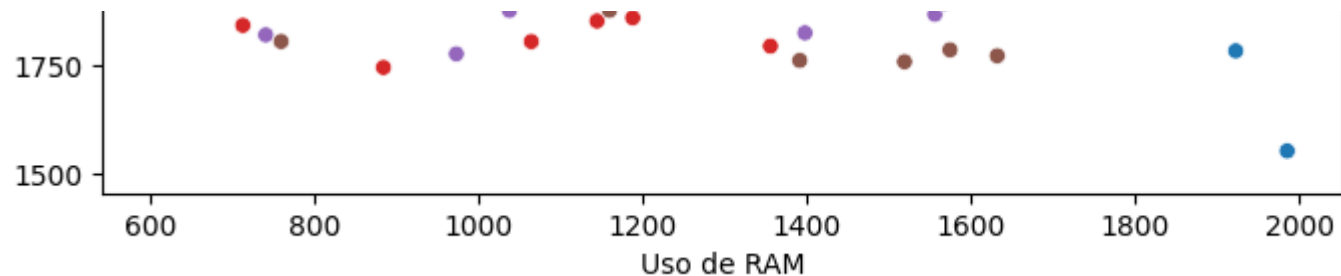
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,6))

ax = sns.scatterplot(
    data=df_resumen,
    x="RAM",
```

```
x= RAM ,  
y="SWAP",  
hue="tratamiento"  
)  
  
ax.legend(  
    bbox_to_anchor=(1.05, 1),  
    loc="upper left"  
)  
  
plt.title("Relación entre uso de la RAM y SWAP ")  
plt.xlabel("Uso de RAM")  
plt.ylabel("Uso de SWAP")  
  
plt.show()
```





Este gráfico evidencia una diferencia interesante en la forma en que Linux y Windows administran la memoria disponible. En el caso de Linux, se observa un comportamiento más flexible respecto al uso de memoria virtual, utilizando swap con mayor facilidad incluso cuando todavía existe memoria RAM disponible.

Esto sugiere un comportamiento adaptativo del sistema, donde Linux parece permitir una mayor utilización de swap como parte de su estrategia de administración de memoria, en lugar de intentar mantener la mayor cantidad posible de información exclusivamente en RAM.

Por otro lado, Windows parece presentar una tendencia más conservadora respecto al uso de memoria virtual, priorizando mantener la información en memoria física antes de recurrir al swap. Aunque esto puede reducir la dependencia del almacenamiento secundario, también podría generar mayor presión sobre la RAM disponible bajo escenarios limitados.

Conclusiones

En resumen, los resultados obtenidos muestran que Ubuntu logró mantener un mayor promedio de FPS y una mayor estabilidad en comparación con Windows 11 bajo las condiciones evaluadas. Asimismo, el comportamiento frente a la reducción de memoria RAM difirió entre ambos sistemas operativos, llegando incluso a presentar tendencias prácticamente opuestas. También se observaron ciertas irregularidades en los tratamientos intermedios, especialmente en el nivel de 3800 MB, las cuales podrían atribuirse a las distintas estrategias de administración y distribución de recursos implementadas por cada sistema operativo.

De igual forma, los resultados sugieren que el enfoque más conservador de Windows respecto al uso de memoria virtual puede

estar asociado con un mayor frametime en comparación con Ubuntu. A su vez, se observó una marcada preferencia por mantener la información en memoria RAM antes que recurrir al swap, evidenciada por el reducido cambio en el uso de memoria virtual entre los tratamientos, con excepción del caso de 3800 MB, donde el comportamiento fue diferente al esperado y podría reflejar una adaptación del sistema ante un nivel específico de presión de memoria.

Por otra parte, Linux mostró un comportamiento más flexible en la utilización de memoria virtual, utilizando el swap con mayor facilidad como parte de su estrategia de administración de memoria. Este comportamiento estuvo acompañado de mejores resultados de rendimiento en los escenarios evaluados, aunque sería necesario realizar estudios adicionales para determinar hasta qué punto esta estrategia es la causa directa de la mejora observada.

En conjunto, los resultados permiten concluir que tanto la reducción de memoria RAM como el sistema operativo influyen en el desempeño de Don't Starve Together Offline, no solo desde la perspectiva del rendimiento medido en FPS, sino también a través de la forma en que cada sistema administra la memoria física y virtual bajo condiciones de recursos limitados.

